

Построение пайплайнов данных

Kubernetes

Малахов Дмитрий Андреевич
Новосельцев Максим Сергеевич

23 января 2026 года

До K8s



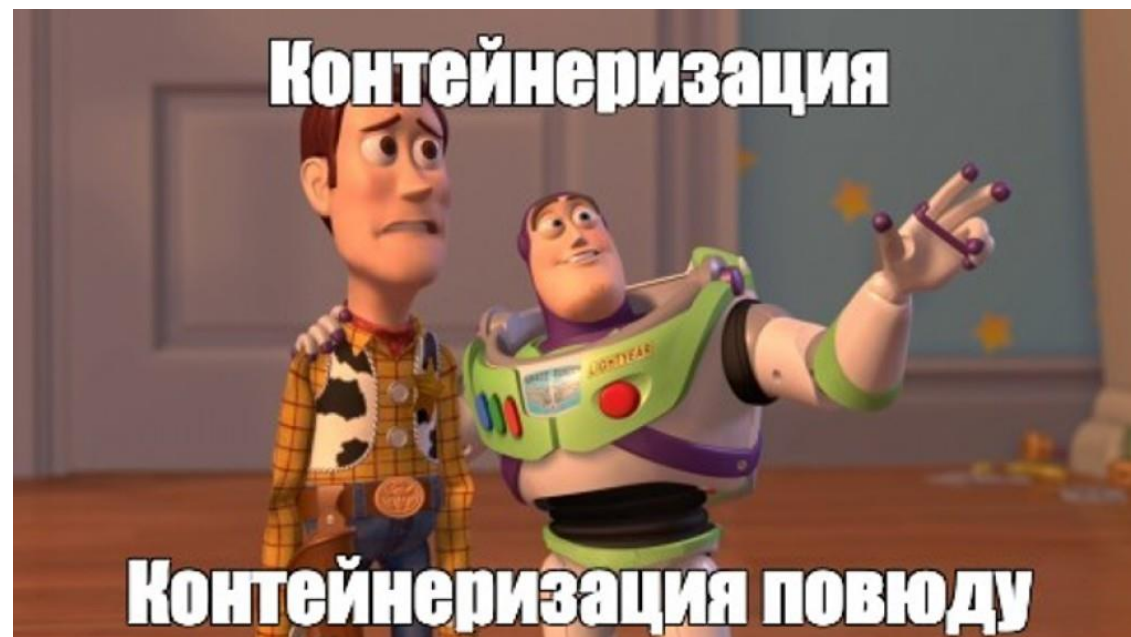
Kubernetes

- это система с открытым кодом которая автоматизирует развертывание, масштабирование и управление контейнеризированными приложениями. Она помогает запускать, масштабировать и поддерживать приложения, упрощая работу с ними и распределяя нагрузку между сервисами.



Решает задачи

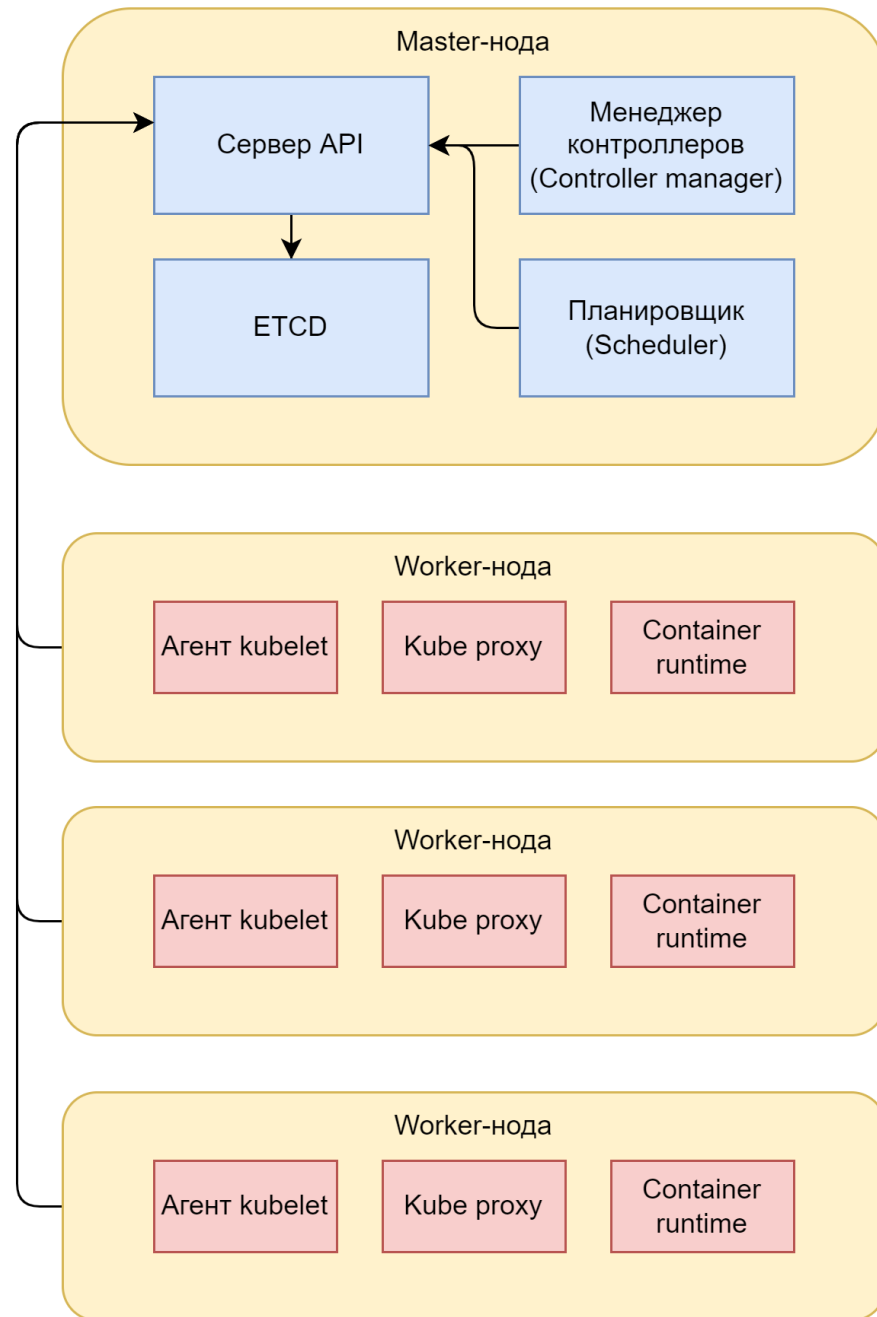
- Деплоймент
- Самовосстановление
- Управление
- Масштабирование
- Балансировка трафика

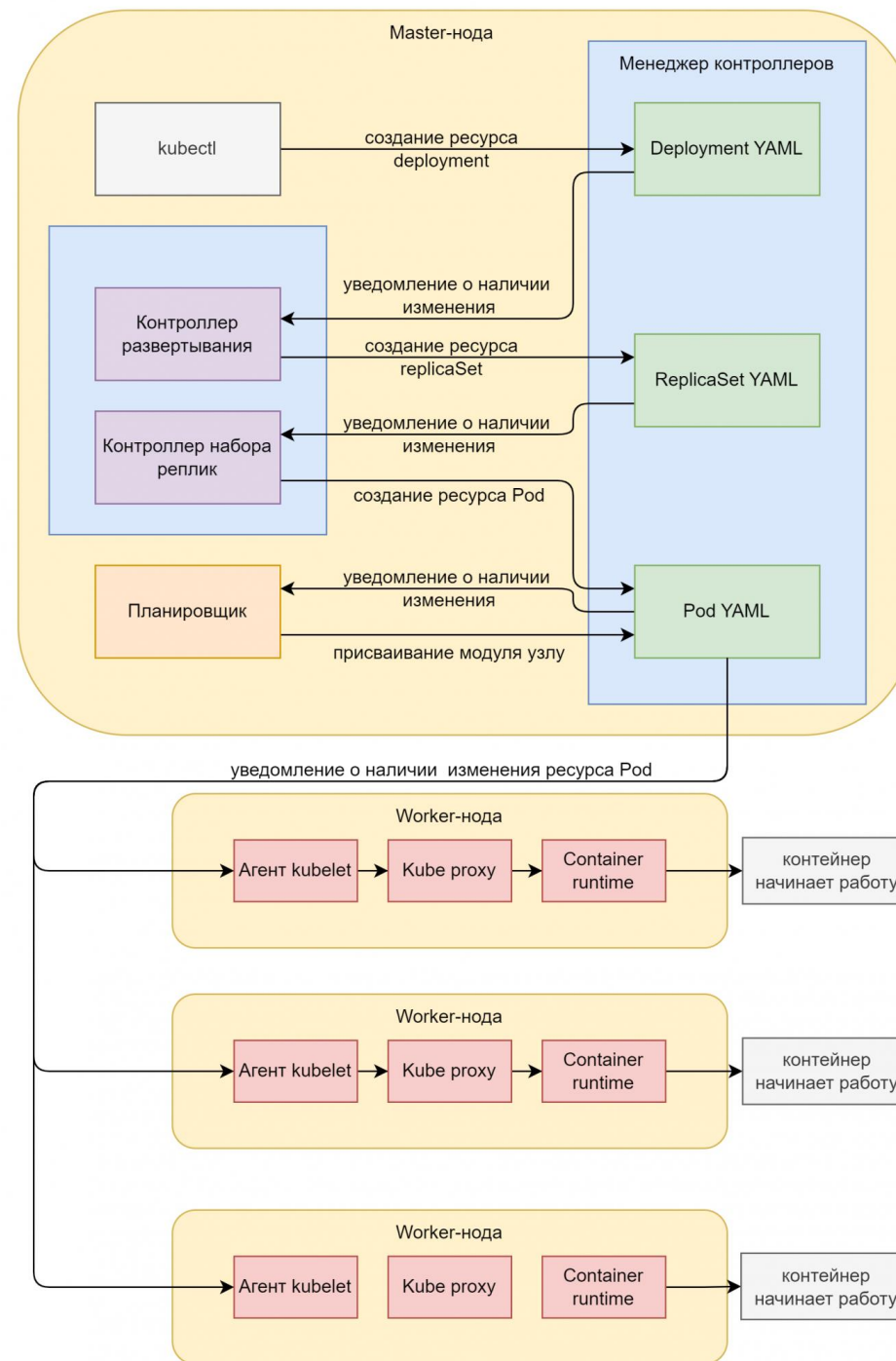


DEEP DIVING



INTO KUBERNETES





Namespaces

```
novms@MacBook-Pro-Maksim:~  
≡ Send Snippet...  
> clear  
> kubectl get namespaces  
NAME                                STATUS    AGE  
argo                                Active    145d  
argo-events                         Active    7d19h  
argocd                              Active    76d  
cattle-fleet-clusters-system        Active    145d
```

```
novms@MacBook-Pro-Maksim:~  
≡ Send Snippet...  
> kubectl get pods -n argo  
NAME                                READY    STATUS    RESTARTS   AGE  
argo-server-855cbc6fb8-fbx5m        1/1      Running   0           4d2h  
workflow-controller-78f64ddfb8-stbtb 1/1      Running   1           4d2h  
~ >
```


YAML конфигурация

```
kind: Pod
apiVersion: v1
metadata:
  name: my-pod
spec:
  containers:
  - name: nginx
    image: nginx:latest
```

Deprecated API Migration Guide

As the Kubernetes API evolves, APIs are periodically reorganized or upgraded. When APIs evolve, the old API is deprecated and eventually removed. This page contains information you need to know when migrating from deprecated API versions to newer and more stable API versions.

Removed APIs by release [↗](#)

v1.32

The **v1.32** release stopped serving the following deprecated API versions:

Flow control resources

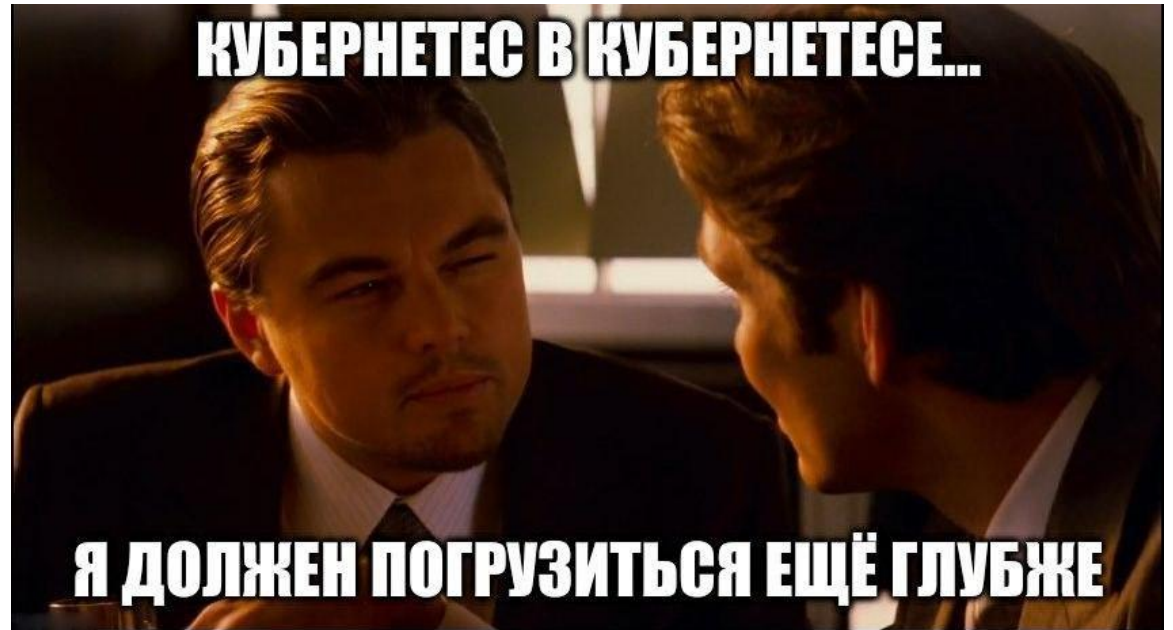
The **flowcontrol.apiserver.k8s.io/v1beta3** API version of FlowSchema and PriorityLevelConfiguration is no longer served as of v1.32.

- Migrate manifests and API clients to use the **flowcontrol.apiserver.k8s.io/v1** API version, available since v1.29.
- All existing persisted objects are accessible via the new API
- Notable changes in **flowcontrol.apiserver.k8s.io/v1**:
 - The PriorityLevelConfiguration `spec.limited.nominalConcurrencyShares` field only defaults to 30 when unspecified, and an explicit value of 0 is not changed to 30.

- v1 для Pod и Service
- apps/v1 для Deployment

Kind

- Pod
- Deployment
- Service
- ConfigMap
- Secret



Metadata

```
metadata:  
  name: my-app  
  namespace: development  
  labels:  
    app: my-app  
    environment: dev
```

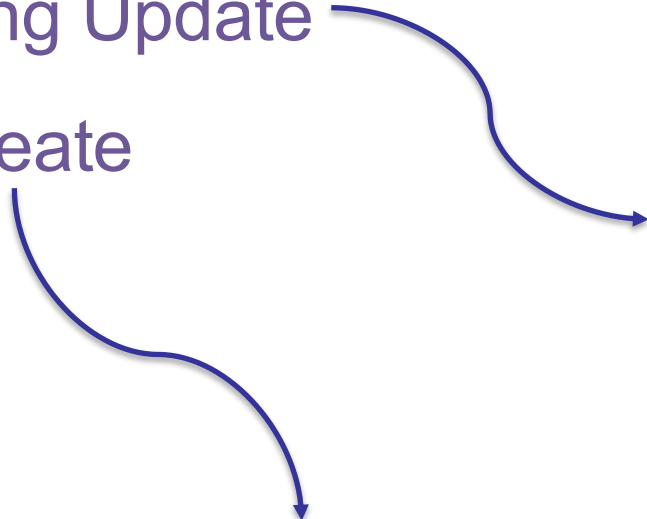
Deployment

- API-объект Deployment
- ReplicaSet
- Поды
- Метки и селекторы
- Сервисы



Стратегии обновления

- Rolling Update
- Recreate



```
spec:  
  strategy:  
    type: RollingUpdate  
    rollingUpdate:  
      maxSurge: 1  
      maxUnavailable: 0
```

```
spec:  
  strategy:  
    type: Recreate
```


Requests & Limits



Requests

— это минимальное количество ресурсов, которое Kubernetes предоставляет конкретному контейнеру.

```
resources:  
  requests:  
    memory: "128Mi"  
    cpu: "750m"
```

Requests

- Запросы должны формулироваться, исходя из фактического использования ресурсов
- Устанавливая запросы, учитывайте, для чего именно предназначена каждая рабочая нагрузка
- Соотносите запросы с тем, сколько ресурсов доступно на данном узле
- Учитывайте, что мощность ЦП может варьироваться

Limits

— это максимальный объём ресурсов конкретного типа, которые может потребить контейнер.

```
resources:  
  limits:  
    memory: "1024Mi"  
    cpu: "1500m"
```

Limits

- Решайте, каким рабочим нагрузкам отдать приоритет
- Учитывайте масштабируемость кластер
- Присваивайте диапазоны лимитов

```
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-resource-constraint
spec:
  limits:
    - default: # this section defines default limits
      cpu: 500m
      defaultRequest: # this section defines default requests
        cpu: 500m
      max: # max and min define the limit range
        cpu: "1"
      min:
        cpu: 100m
    type: Container
```

Проблема	Причина	Способы исправления
Не удаётся распланировать поды, поскольку запросы слишком высоки.	Не хватает ресурсов, чтобы удовлетворить все запросы.	• Снизить запросы • Добавить узлы
Поды вытесняются или уничтожаются командой OOMKill	Из-за высокого потребления ресурсов возникает их дефицит, из-за чего приходится либо перепланировать поды, либо уничтожать некоторые из них.	• Устанавливайте лимиты, чтобы предотвратить избыточное потребление ресурсов некоторыми контейнерами. • Устанавливайте диапазоны лимитов, чтобы справиться с потреблением ресурсов в пределах пространства имён. Добавляйте узлы.
Узлы перегружены или не отвечают	Ресурсы ЦП или памяти узла используются по максимуму, поскольку контейнерам их требуется слишком много.	• Устанавливайте лимиты, чтобы ограничить потребление ресурсов контейнерами. • Пользуйтесь DaemonSets, с их помощью перемещайте самые требовательные контейнеры на те узлы, где больше ресурсов.
Наблюдается падение производительности приложений или возникают проблемы, связанные с задержками.	Если ресурсов явно недостаточно, то возникают проблемы с производительностью приложений	• Устанавливайте такие запросы, которые гарантируют адекватное количество ресурсов • Устанавливайте лимиты, которые не позволят приложениям отобрать у соседей слишком много ресурсов
Неожиданные всплески потребления ресурсов (могут объясняться резким увеличением трафика по конкретной рабочей нагрузке)	Из-за багов в приложениях (например, из-за утечек памяти) иногда случается резкий всплеск потребления ресурсов	• Устанавливайте лимиты, чтобы ограничить, сколько ресурсов может использовать то или иное приложение. • Выявляйте и исправляйте те базовые проблемы в приложениях, которые и провоцируют непреднамеренно высокое потребление ресурсов.

ReadinessProbe

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod-readinessprobe
spec:
  containers:
  - name: nginx
    image: nginx
    readinessProbe:
      httpGet:
        path: /ready
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5
```

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: myapp-container
    image: myapp
    readinessProbe:
      exec:
        command:
        - cat
        - /tmp/ready
      initialDelaySeconds: 5
      periodSeconds: 5
```

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: myapp-container
    image: myapp
    readinessProbe:
      tcpSocket:
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5
```


LivenessProbe

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod-livenessprobe
spec:
  containers:
  - name: nginx
    image: nginx
    livenessProbe:
      httpGet:
        path: /health
        port: 80
      initialDelaySeconds: 15
      periodSeconds: 20
```

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: myapp-container
    image: myapp
    livenessProbe:
      exec:
        command:
        - cat
        - /tmp/healthy
      initialDelaySeconds: 15
      periodSeconds: 20
```

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: myapp-container
    image: myapp
    livenessProbe:
      tcpSocket:
        port: 80
      initialDelaySeconds: 15
      periodSeconds: 20
```

StartupProbe

```
apiVersion: v1
kind: Pod
metadata:
  name: slow-start-app
spec:
  containers:
  - name: myapp
    image: myapp
    startupProbe:
      httpGet:
        path: /start
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
```

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: myapp-container
    image: myapp
    startupProbe:
      exec:
        command:
        - cat
        - /tmp/start
      failureThreshold: 30
      periodSeconds: 10.
```

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: myapp-container
    image: myapp
    startupProbe:
      tcpSocket:
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
```

ConfigMap

— это объект Kubernetes, который используется для хранения неконфиденциальных данных конфигурации в виде пар «ключ — значение». Он позволяет разделить конфигурацию приложения и код, что делает приложение более гибким и удобным в управлении.

ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  service_url: "https://api.example.com"
  timeout: "30"
```

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mycontainer
    image: myapp:1.0
    env:
    - name: SERVICE_URL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: service_url
    - name: TIMEOUT
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: timeout
```

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mycontainer
    image: myapp:1.0
    volumeMounts:
    - name: config-volume
      mountPath: /etc/config
  volumes:
  - name: config-volume
    configMap:
      name: app-config
```

Secret

— это объект Kubernetes, предназначенный для хранения чувствительных данных, таких как пароли, ключи шифрования и API-токены. Он позволяет безопасно передавать такие данные в Pod'ы, защищая их от утечек и неправильного доступа.

Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: api-secret
data:
  api_key: dXNlcm5hbWU6cGFzc3dvcmQ=
```

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mycontainer
    image: myapp:1.0
    env:
    - name: API_KEY
      valueFrom:
        secretKeyRef:
          name: api-secret
          key: api_key
```

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mycontainer
    image: myapp:1.0
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secret
  volumes:
  - name: secret-volume
    secret:
      secretName: api-secret
```

Лучшая практика

- Разделение конфигурации и кода
- Использование нескольких окружений
- Минимизация данных
- Шифрование секретов
- Ограничение доступа
- Частая ротация ключей

Kubernetes Volumes

- Хранить промежуточные или постоянные данные
- Делиться файлами между контейнерами внутри одного пода
- Иметь доступ к «общим» данным, разделяемым между разными подами
- Гарантировать сохранение данных после выхода из строя (рестарта) контейнера или пода

emptyDir

```
apiVersion: v1
kind: Pod
metadata:
  name: example-emptydir
spec:
  containers:
    - name: busybox
      image: busybox
      command: ["sleep", "3600"]
      volumeMounts:
        - name: cache-volume
          mountPath: /cache # Монтируем volume в /cache
  volumes:
    - name: cache-volume
      emptyDir: {} # Тип тома emptyDir
```

hostPath

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  containers:
    - name: test-container
      image: nginx
      volumeMounts:
        - name: host-volume
          mountPath: /data # В контейнере это /data
  volumes:
    - name: host-volume
      hostPath:
        path: /mnt/data # А на хостовой машине папка /mnt/data
        type: Directory
```

persistentVolumeClaim (PVC)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-example
spec:
  capacity:
    storage: 1Gi # Объем PV
  accessModes:
    - ReadWriteOnce # Только один под может писать и читать
  hostPath:
    path: /mnt/pvdata # Физическое хранилище на ноде
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-example
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 500Mi # Нам нужно 500Mб
```

persistentVolumeClaim (PVC)

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-pvc
spec:
  containers:
    - name: app
      image: busybox
      command: ["sleep", "3600"]
      volumeMounts:
        - name: storage
          mountPath: /data # Примонтировали volume в /data
  volumes:
    - name: storage
      persistentVolumeClaim:
        claimName: pvc-example # Связываем с PVC
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: test-app
  template:
    metadata:
      labels:
        app: test-app
    spec:
      containers:
        - name: app
          image: my-app:latest
          volumeMounts:
            - name: app-storage
              mountPath: /usr/share/app/data
      volumes:
        - name: app-storage
          persistentVolumeClaim:
            claimName: app-pvc
```

StorageClass

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: kubernetes.io/aws-ebs # Провайдер хранилища (зависит от облака)
parameters:
  type: gp2
reclaimPolicy: Delete
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-fast-claim
spec:
  storageClassName: fast-storage
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Метки

— это пары ключ-значение, которые добавляются к таким объектам, как поды. Метки предназначены для идентификации атрибутов объектов, которые имеют значимость и важны для пользователей, но при этом не относятся напрямую к основной системе.

```
apiVersion: v1
kind: Pod
metadata:
  name: label-demo
  labels:
    environment: production
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```

Условие набора

Условие меток на основе набора фильтрует ключи в соответствии с набором значений. Поддерживаются три вида операторов: `in`, `notin` и `exists`.

```
environment in (production, qa)
tier notin (frontend, backend)
partition
!partition
```

```
kubectl get pods -l 'environment in (production, qa)'
```

Условие набора

Условие меток на основе набора фильтрует ключи в соответствии с набором значений. Поддерживаются три вида операторов: `in`, `notin` и `exists`.

```
environment in (production, qa)
tier notin (frontend, backend)
partition
!partition
```

```
selector:
  matchLabels:
    component: redis
  matchExpressions:
    - {key: tier, operator: In, values: [cache]}
    - {key: environment, operator: NotIn, values: [dev]}
```

```
kubectl get pods -l 'environment in (production, qa)'
```


Сервис

— это объект, который используется для определения логических групп подов и правил доступа к ним.

- ClusterIP
- NodePort
- LoadBalancer
- ExternalName
- Headless Service

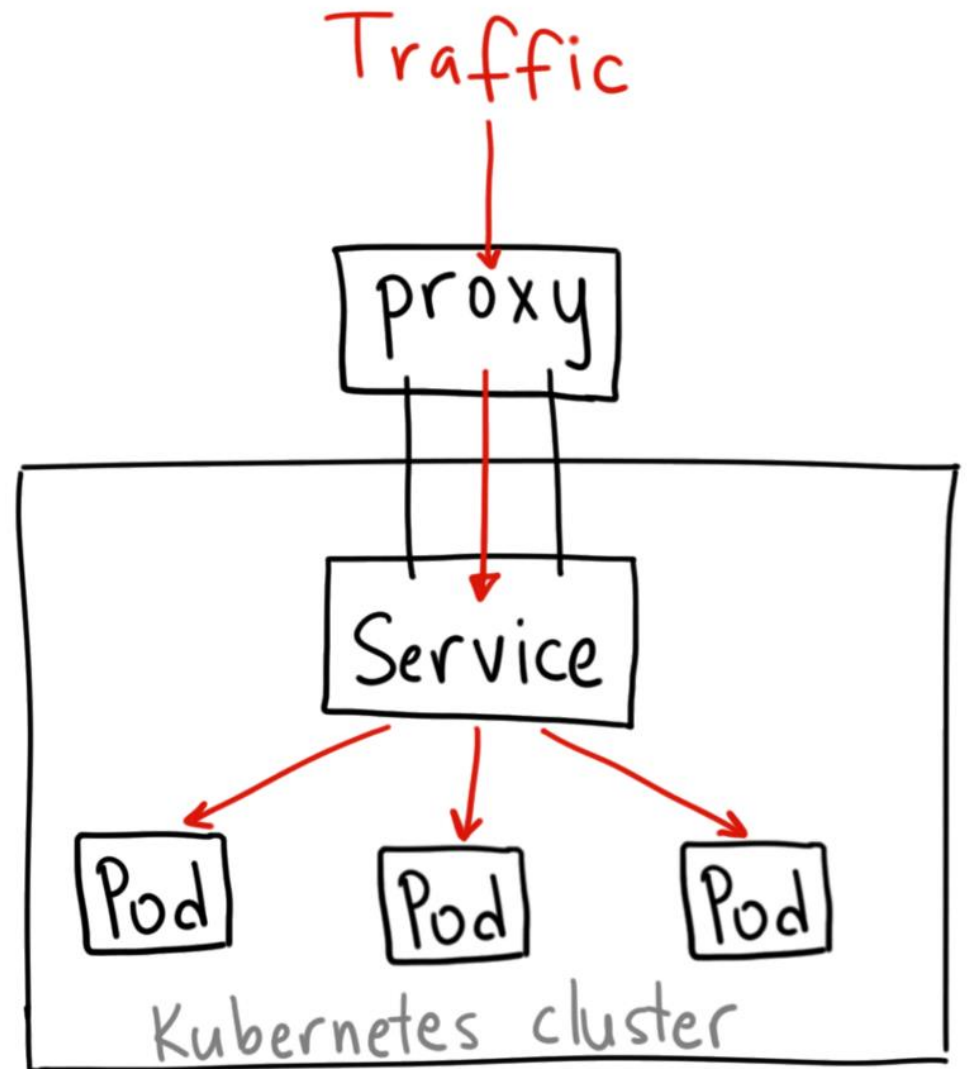
Сервис

— это объект, который используется для определения логических групп подов и правил доступа к ним.

- ClusterIP
- NodePort
- LoadBalancer

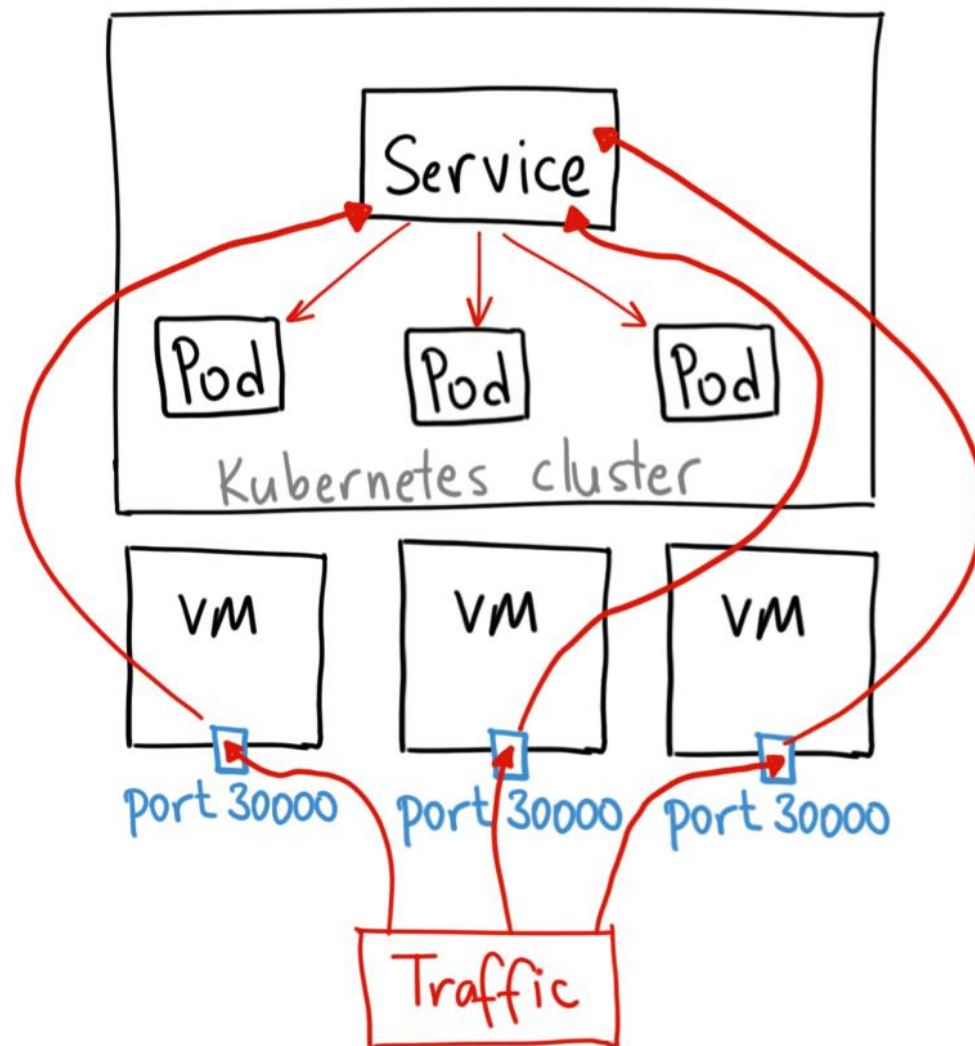
ClusterIP

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-clusterip-service
  namespace: service-clusterip
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```



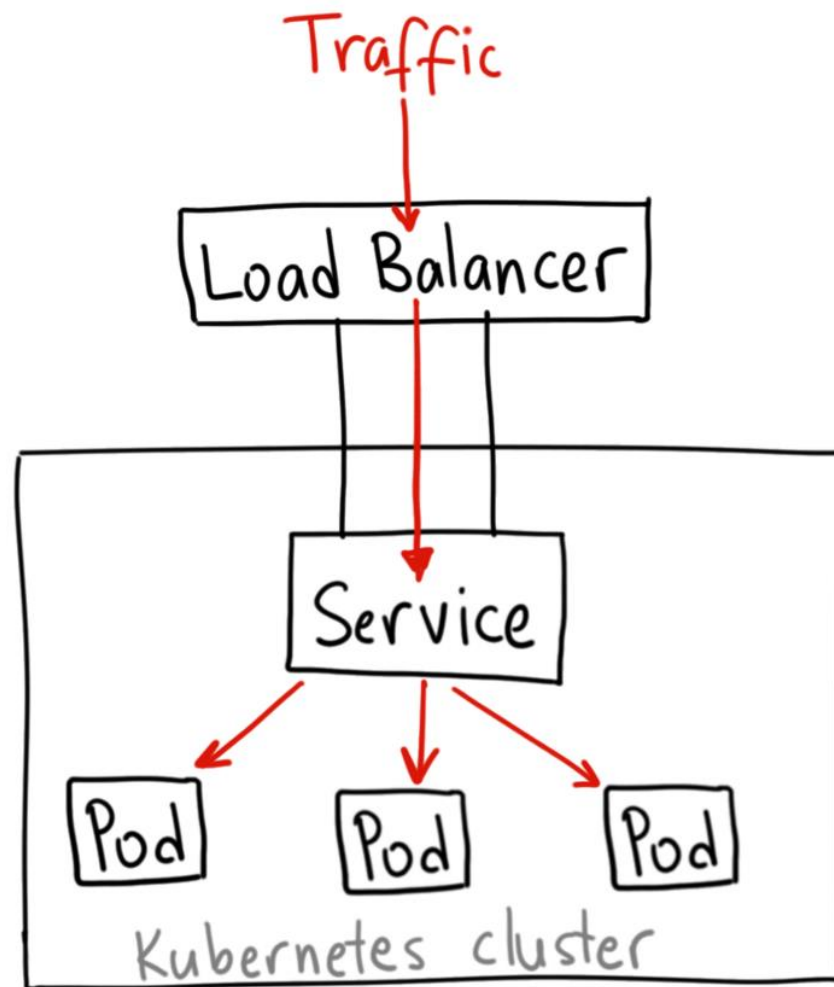
NodePort

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport-service
  namespace: service-nodeport
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
  type: NodePort
```

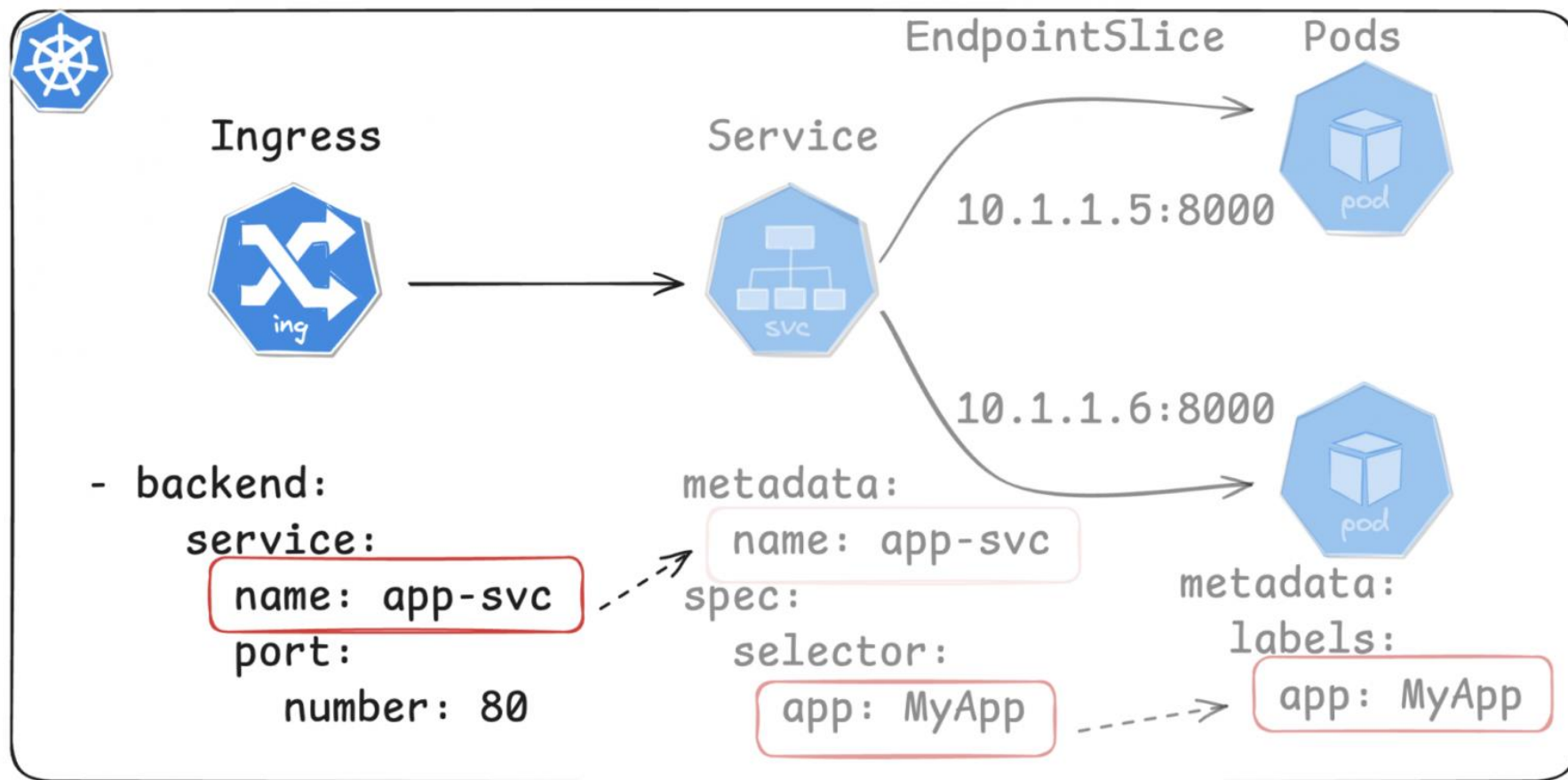


LoadBalancer

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport-service
  namespace: service-nodeport
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
  type: NodePort
```



Ingress



RBAC

- Role
- RoleBinding
- ClusterRole
- ClusterRoleBinding
- ServiceAccount



ServiceAccount

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cicd-deployer
  namespace: deployment
```


Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: prod
  name: config-secret-manager
rules:
- apiGroups: [""]
  resources: ["configmaps", "secrets"]
  verbs: ["get", "list", "create", "update", "delete"]
```

RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: "janedoe"
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

ClusterRole

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: service-manager
rules:
- apiGroups: [""]
  resources: ["services"]
  verbs: ["get", "list", "create", "update", "delete"]
```

ClusterRoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: read-nodes-global
subjects:
- kind: User
  name: "johndoe"
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```

Домашнее задание

Сравнительный анализ инструментов для управления секретами в Kubernetes

Вам нужно провести исследование и сравнить как минимум 3 инструмента для работы с секретами в Kubernetes (например, native Secrets, HashiCorp Vault и др.).

Что нужно сделать:

Самостоятельно определите критерии сравнения — выберите те аспекты, которые считаете важными для реального проекта (безопасность, удобство разработки, производительность, audit, ротация секретов, интеграция с GitOps, стоимость внедрения и т.д.).

Проведите анализ каждого инструмента по выбранным критериям, опишите сильные и слабые стороны, рассмотрите разные сценарии использования